Kurtosys Studio Product-Page Monorepo — User Guide

Repository: `ksys-studio-pp-template` · Companion widget starter: `ksys-studio-widget-template`

This guide explains how to reuse this monorepo for a new client, add or rename apps, develop locally, build for Studio, and follow the accessibility / security defaults that ship with every widget.

Contents

1. What this repo is
2. Prerequisites
3. Repository layout
4. Start a new client project
5. Understanding `studio.config.json`
6. Manage apps (list / add / rename)
7. Develop a single app
8. Configure environments (GUIDs & tokens)
9. DXM mount contract
10. Build for dev / staging / prod
11. Built-in defaults (a11y, Tailwind, embed, security)
12. Security audit reports
13. End-to-end checklist
14. Troubleshooting
15. Regenerate this PDF

1. What this repo is

A **generic, reusable monorepo** for Kurtosys Studio / DXM embeddable React widgets. Each widget is its own Vite app under `development_v1/product-page-apps/`, discovered and orchestrated by `studio.config.json` and root npm scripts.

Starter apps included out of the box:

App id	Folder (default client)	Mount id	Embed global	Dev port
`pageHeader`	`ksys-client-pageHeader`	`#pageHeader`	`window.pageHeader`	5173
`overview`	`ksys-client-overview`	`#overview`	`window.overview`	5174
`performance`	`ksys-client-performance`	`#performance`	`window.performanceApp` *	5175
`portfolioBreakdown`	`ksys-client-portfolioBreakdown`	`#portfolioBreakdown`	`window.portfolioBreakdown`	5176
`literature`	`ksys-client-literature`	`#literature`	`window.literature`	5177

> * The browser already owns `window.performance`. For that app id the embed loader automatically uses `performanceApp` so the native Performance API is not overwritten.

2. Prerequisites

- **Node.js** 20+ and **npm**
- **Python 3.10+** (security reports and PDF generation)
- Optional: Chrome / Edge / Chromium for PDF export if `xhtml2pdf` is unavailable
- Sibling folder `ksys-studio-widget-template` (used by `npm run apps:add`)

3. Repository layout

```
ksys-studio-pp-template/
├── USER_GUIDE.html / USER_GUIDE.pdf   ← this guide
├── README.md
├── studio.config.json                ← app registry + client slug
├── package.json                      ← monorepo orchestration scripts
├── docs/ACCESSIBILITY.md
├── scripts/
│   ├── add-app.mjs
│   ├── rename-app.mjs
│   ├── set-client.mjs
│   ├── list-apps.mjs
│   ├── build-all.mjs
│   ├── run-app.mjs
│   └── run-all.mjs
├── tools/
│   ├── security_audit_report.py
│   ├── build_user_guide_pdf.py
│   └── pdf_from_html.py
├── BuildResult/{dev,staging,prod}/   ← collected production-like bundles
├── reports/                          ← security audit HTML/PDF
└── development_v1/product-page-apps/
    ├── config/                       ← optional shared client utilities
    └── ksys-<client>-<appId>/        ← one Vite app per widget
```

4. Start a new client project

1. Copy the template (keep the widget template nearby):

   ```
   cp -R ~/Documents/GitHub/ksys-studio-pp-template ~/Documents/GitHub/acme-pp
   # ensure widget template remains at:
   # ~/Documents/GitHub/ksys-studio-widget-template
   cd ~/Documents/GitHub/acme-pp
   ```

2. Set the client slug (renames all `ksys-client-*` folders and updates config):

   ```
   npm run client:set -- acme
   ```

   Folders become `ksys-acme-pageHeader`, `ksys-acme-overview`, etc.

3. Confirm registry:

   ```
   npm run apps:list
   ```

4. Fill GUIDs / API tokens in each app's `config/env.config.cjs` (see §8).
5. Run an app:

```
npm run dev -- overview
```

> **Do not** keep production widgets only outside this monorepo. Always register them under `development_v1/product-page-apps/` so builds and audits discover them.

5. Understanding `studio.config.json`

This file is the source of truth for orchestration:

```
{
  "clientSlug": "client",
  "appPrefix": "ksys",
  "appsRoot": "development_v1/product-page-apps",
  "buildResultRoot": "BuildResult",
  "widgetTemplatePath": "../ksys-studio-widget-template",
  "apps": [
    { "id": "pageHeader", "label": "Page Header", "port": 5173 },
    { "id": "overview", "label": "Overview", "port": 5174 }
  ]
}
```

- `clientSlug` — folder segment after `ksys-`
- `apps[].id` — camelCase app id (mount id, JS filename, embed global)
- `apps[].port` — Vite dev server port
- `widgetTemplatePath` — path to the single-app scaffold used by `apps:add`

6. Manage apps (list / add / rename)

List

```
npm run apps:list
```

Add a new app

```
npm run apps:add -- holdingsExposure 5178
```

This will:

- Copy `ksys-studio-widget-template` into `ksys-<client>-holdingsExposure/`
- Set `filename`, `containerId`, and `embedGlobalName` to `holdingsExposure`
- Assign the Vite port and register the app in `studio.config.json`

Rename an app

```
npm run apps:rename -- literature materials
```

Renames the folder, package name, mount id, embed global references, and config entry.

Change client slug later

```
npm run client:set -- newclient
```

7. Develop a single app

```
# from monorepo root
npm run dev -- overview

# equivalent:
cd development_v1/product-page-apps/ksys-<client>-overview
npm install   # first time only
npm run dev
```

Open the URL Vite prints (default `http://localhost:5174` for overview). Local `index.html` simulates the Kurtosys App Manager + mount node.

Useful per-app scripts:

Script	Purpose
`npm run dev`	Vite HMR development server
`npm run test`	Vitest unit tests
`npm run lint`	ESLint (includes jsx-a11y)
`npm run build:dev`	Production bundle using development env config
`npm run build:staging`	Staging env config
`npm run build:prod`	Production env config
`npm run audit:deps`	Write `npm-audit.json` (omit dev deps)

From the monorepo root you can also run tests/lints across all registered apps:

```
npm test
npm run lint
```

8. Configure environments (GUIDs & tokens)

Each app has `config/env.config.cjs` with three tiers: `development`, `staging`, `production`.

Replace placeholders before deploy:

- `guid` — Studio applicationCode UUID
- `apiToken` / `savedSearchToken` — public Kurtosys tokens for that tier
- `filename` — output JS name (e.g. `overview.js`)
- `containerId` — DXM mount element id
- `embedGlobalName` — `window.<name>` for the embed loader

> Do not commit real secrets to public remotes. Prefer CI secrets or a local untracked override when sharing the repo.

9. DXM mount contract

On the DXM / Elementor page (or local `index.html`):

```
<div
  data-ksys-app-template-id="ksys-app-manager"
  data-input-client-code="YOUR_CLIENT_CODE"
  data-input-culture="en-US"
  data-input-product-type="mutualFunds"
></div>

<div id="overview"></div>
<script src=".../overview.js"></script>
```

Each widget resolves inputs in this order:

1. Mount node `data-input-*` attributes
2. App Manager node attributes
3. Query string fallbacks (`kurtosys_id`, `clientCode`) for local/dev

Implementation lives in `src/utils/resolve-mount-inputs.ts` and `src/main.tsx` (find container → create React root → MutationObserver guard).

10. Build for dev / staging / prod

From the monorepo root:

```
npm run build:dev
npm run build:staging
npm run build:prod
```

For each registered app this:

1. Runs the matching `build:*` npm script
2. Produces a flat IIFE bundle with CSS inlined into the JS
3. Copies **only the relevant artifact** into `BuildResult/<env>/<appId>/<appId>.js`

> Scaffolding files (`index.html` , manifests, `embed-loader.js` , `vite.svg`) stay in each app's `dist-build/` for local inspection but are not copied into `BuildResult` by `build-all` — matching the clean deploy layout used on BP.

Upload the JS from `BuildResult/<env>/<appId>/` to the matching Studio application asset.

11. Built-in defaults

Accessibility (WCAG 2.1 / 2.2 oriented)

- Semantic landmarks and labelled headings in the starter widget
- High-contrast CSS variables (`--studio-color-text` on white)
- Visible `:focus-visible` rings for keyboard users
- `VisuallyHidden` + `accessibleImageProps` helpers
- `prefers-reduced-motion` respect
- `eslint-plugin-jsx-a11y` recommended rules

See `docs/ACCESSIBILITY.md` for the checklist when building real UI.

Tailwind specificity

```
@import "tailwindcss" important;
```

Every utility is emitted with `!important` so DXM / Elementor host styles cannot silently override your layout.

Embed loader global name

`scripts/generate-embed-loader.cjs` exposes `window.<appId>` (for example `window.overview`), not `YourAppEmbed` or a hard-coded client name. Reserved browser globals (like `performance`) are automatically remapped.

Security dependency pins

Each app pins / overrides the previously vulnerable set:

- `vite`
- `rollup`
- `tar`
- `picomatch`
- `postcss`

12. Security audit reports

```
pip install -r tools/requirements-audit-report.txt   # optional PDF helper
npm run audit:security
```

Produces timestamped HTML + PDF under `reports/`.

Scope is intentional and production-facing:

- `npm audit --omit=dev` for every discovered `ksys-*` app package
- Heuristic secret / leak scan of app `dist-build` and `BuildResult` JS only

> This is not a penetration test. Validate findings manually before acting on them.

13. End-to-end checklist

1. Copy monorepo → `npm run client:set -- <slug>`
2. Add/rename apps as needed → `npm run apps:list`
3. Fill `config/env.config.cjs` for each app / environment
4. Implement UI in `src/components/Widget/` (keep a11y patterns)
5. `npm run dev -- <appId>` and verify mount + inputs
6. `npm test` / `npm run lint`
7. `npm run build:staging` (or prod) → check `BuildResult/`
8. Upload JS assets to Studio
9. Wire DXM containers to matching mount ids
10. `npm run audit:security` before release

14. Troubleshooting

Symptom	Likely fix
Mount container not found	Ensure DXM element id matches `containerId` / `VITE_CONTAINER_ID`
`apps:add` cannot find template	Confirm `widgetTemplatePath` in `studio.config.json` points at `ksys-studio-widget-template`
Styles overridden by DXM	Confirm `@import "tailwindcss" important;` is present in `src/index.css`
Embed global missing / wrong	Check `embedGlobalName` in `env.config.cjs`; rebuild so `embed-loader.js` regenerates
`window.performance` broken after load	Use `performanceApp` (default) — never set embed global to reserved browser names
Build env GUIDs still placeholders	Replace `YOUR_*_GUID_HERE` before Studio deploy
Python 3.9 fails on tooling	Use Python 3.10+ (type syntax / tooling expectation)

15. Regenerate this PDF

Edit `USER_GUIDE.html`, then from the monorepo root:

```
python3 tools/build_user_guide_pdf.py
# or
npm run docs:user-guide
```

PDF generation tries **xhtml2pdf** first, then headless Chrome / Edge / Chromium. The HTML guide is always the source of truth if PDF generation is unavailable.

Kurtosys Studio product-page monorepo template · User guide · Keep this file with the repo so every client handoff includes the same operating instructions.